Securing The Chink in Kerberos' Armor, FAST! Understanding The Need For Kerberos Armoring

trimarcsecurity.com/hub-post/securing-the-chink-in-kerberos-armor-fast-understanding-the-need-for-kerberos-armoring

Darryl Baker

July 16, 2024

Updated: Dec 13, 2024

Introduction



In 1989, the first public version of Kerberos was released (v4). 35 years later, Kerberos is up to version 5, and attacks against the protocol are more frequent than ever. These attacks are so common because Kerberos was originally created in a different era of network security. As attacks have become more sophisticated over the years, more sophisticated security controls needed to be implemented with the protocol.

Fortunately, Microsoft's implementation of Kerberos in Active Directory (AD) includes mechanisms to help mitigate some of these threats. This blog will discuss one of these mechanisms known as Kerberos Armoring. Kerberos Armoring was introduced in Windows Server 2012 and is supported by all newer versions of Windows Server as well as Windows 8 and later.

What is Kerberos Armoring?

Kerberos armoring, also known as FAST (Flexible Authentication Secure Tunneling), is an extension to the Kerberos protocol that improves the security of the Kerberos authentication process as well as service ticket retrieval. As James Forshaw summarized FAST, “Makes it harder to tamper with Kerberos network traffic.” It involves creating a secure tunnel between a client and the Key Distribution Center (KDC) using an armored key. An armored key is an additional encryption key that is created by the KDC during an initial authentication from the host machine account to secure Kerberos message exchange.

Without Kerberos armoring, the Kerberos authentication process and service ticket request process are exchanged in clear text. This includes Authentication Service Requests (AS-REQ), Authentication Service Responses (AS-REP), Service Ticket Requests (TGS-REQ), and Service Ticket Responses (TGS-REP). While portions of the messages exchanged are encrypted such as the Ticket Granting Ticket (TGT) and session key (to be used for subsequent communications), the exchange of these messages is plainly visible.

The problem with this scenario is that attackers can capture these communication exchanges, then attempt to manipulate the tickets or crack the encrypted pieces of the responses to derive account passwords. User credentials during Kerberos authentication, as well as service account credentials during service ticket responses, are often targeted this way via well know methods such as ASREPROASTing and Kerberoasting. The process of attempting to crack a password using one of these methods is completely offline, meaning an attacker can take as much time as needed (assuming that the password of the targeted account hasn’t changed).

Here is a quick breakdown of the inherent problem when Kerberos Armoring is not in place:

Authentication Service (AS) Exchange: Without armoring, the initial exchange between the client and the Authentication Service (AS) is not encrypted. This includes the client's request for a Ticket Granting Ticket (TGT) and the AS's response containing the encrypted TGT. Without Armoring, this exchange could potentially be intercepted by attackers, allowing them to capture the TGT. This also gives attackers the opportunity to attempt to crack the password of the account initiating the Kerberos exchange, as the AS-REP is encrypted with that account’s password.

Ticket Granting Service (TGS) Exchange: The exchange between the client and the Ticket Granting Service (TGS) for obtaining service tickets is also unencrypted by default. This includes the client's request for a service ticket and the TGS-REP containing the encrypted service ticket. Without armoring, this exchange could also be intercepted and potentially used by attackers to crack the password of the service account of the requested resource.

Keep in mind, the content of the Kerberos tickets themselves, including the TGT and service tickets, is encrypted using keys shared between the client and the Key Distribution Center (KDC). However, without armoring, the transmission of these tickets during the authentication process is not encrypted, leaving them vulnerable to attacks.

How Does it Work (high level)?

First, let's talk about the normal Kerberos authentication process (with Pre-authentication in place).

1. When a client initiates the Kerberos authentication process, the client will send its cname (sAMAccountName) along with a timestamp that is encrypted using the client's secret key (derived using a hash of the client's password).

2. The KDC receives the request, and the Authentication Service validates 3 pieces of information:

a. The client account exists in Active Directory.

b. The timestamp was encrypted using the client account's secret key (AD has access to the client's secret).

c. The timestamp is within 5 minutes of the KDC receiving the AS-REQ.

3. If this all checks out, the AS will send the client an encrypted Ticket Granting Ticket (encrypted using the secret key of the krbtgt account) as well as other required session info. This entire response is encrypted using the client's password.

4. When the client requests a service ticket using it's TGT, the KDC will send the client the requested service ticket (encrypted using the service account password).

When armoring is in place, this entire exchange is encrypted using the armor key. So, where does the armor key come from? The armor key is a cryptographic key derived from the negotiated session key during the Client-Server Key Exchange Protocol (C/S KDC-REQ).

If the client is authenticating from a host machine that is joined to the same domain as the client, the host machine's computer account will request a TGT from the KDC using the normal AS process. The host then initiates the Client-Server key exchange which involves the KDC creating a new key using session information, client information, server information, and some mathematical randomness. The armor key is then sent to the host machine (encrypted using the current session key). The client continues with the Kerberos authentication process starting with an AS-REQ. Now, however, all communication between the client and KDC is encrypted using the new armor key obtained by the host.

If the client's domain is different than the domain of the host machine, the host obtains a cross-realm TGT and uses that to request an armor key.

If the host machine cannot obtain a host ticket (potentially because it's not domain joined), but the KDC has an asymmetric signing key (Active Directory Certificate Services) that can be verified by the client, the client can use anonymous PKINIT [RFC6112] [RFC4556] to authenticate the KDC and obtain an anonymous TGT to use to request and armor key. The host machine can also use a cross-realm TGT obtained from an initial TGT obtained using anonymous PKINIT.

What Kerberos Armoring Prevents:

Kerberos Armoring's main goal is to prevent offline dictionary attacks against Kerberos messages during ticket exchange. Armoring also defends against replay and Machine-in-the-Middle attacks. Without armoring, Kerberos tickets could potentially be intercepted and replayed by attackers, allowing unauthorized access to resources. This attack is prevented because even if the attacker acquires a Kerberos ticket, they won't have the necessary armor key to communicate with the KDC. The additional layer of encryption that armoring provides also helps to prevent Kerberos ticket modification, ensuring the integrity of the authentication process. Data confidentiality is also increased as the entire communication between the client and KDC is encrypted.

In September 2022, two vulnerabilities were addressed in Kerberos for all supported Windows Server versions. These vulnerabilities could lead to successful Adversary in the Middle (AitM) attacks and subsequent SYSTEM privileges when Kerberos Armoring is not enabled:

- CVE-2022-33647 Windows Kerberos Elevation of Privilege Vulnerability
- CVE-2022-33679 Windows Kerberos Elevation of Privilege Vulnerability

In reference to CVE-2022-33679, for a user object to be vulnerable, it must be configured with "**Do not require Kerberos preauthentication"** and must have been configured with an RC4 key.

For more in-depth examples of attacks that are stopped by Kerberos Armoring, check out this blog from TrustedSec: https://www.trustedsec.com/blog/i-wanna-go-fast-really-fast-like-kerberos-fast

Configuring Kerberos Armoring

To use Kerberos Armoring, all domain controllers in the domain must be running Windows Server 2012 or higher, the Domain Functional Level (DFL) needs to be Windows Server

2012 or higher, and client devices need to run Windows 8 or higher. Keep in mind that implementing Kerberos Armoring should be done methodically and with a plan:

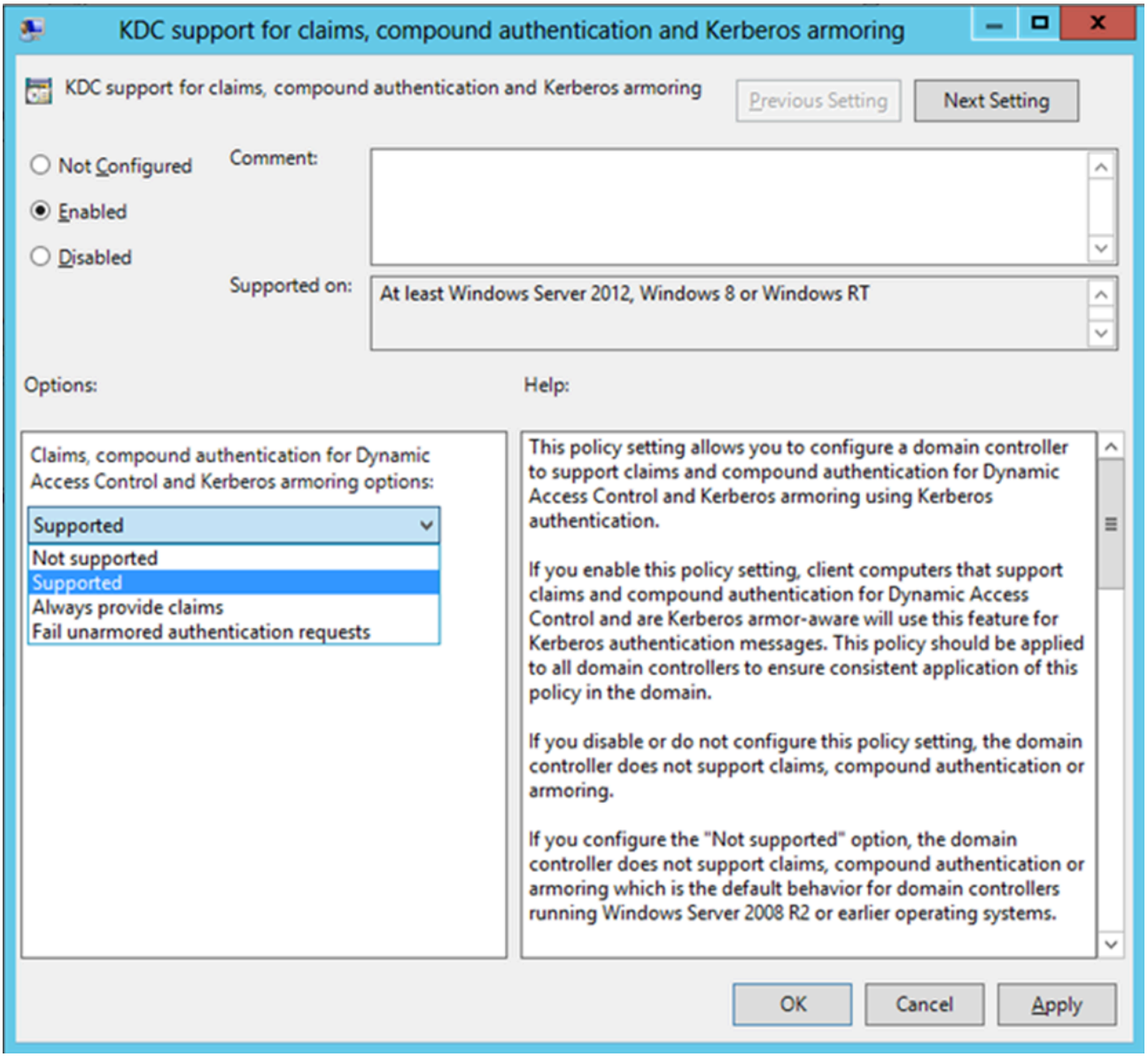
- Ensure that Kerberos Armoring is enabled for clients first. This should also usually done in groups so that that further testing can be done (incase something breaks). This change needs to be applied to all clients before being configured on Domain Controllers.
- Once Armoring is supported on domain clients, configure Kerberos Armoring on Domain Controllers. It is good practice to initially configure Domain Controllers to “Supported”. This will allow for armoring to be used when available, but not fail authentication if armoring is not available.
- Clients that authenticate to Domain Controllers without using Kerberos Armoring will trigger Event ID: 4771 - Kerberos Pre-Authentication Failed. This log will include Failure Code: 0x19 - "Additional pre-authentication required." This indicates that the KDC required Kerberos armoring (FAST), but the client did not use it or did not support it.

Kerberos Armoring can be easily deployed using Group Policy. There are two policy options that must be configured. One to configure armoring on Domain Controllers, and one to configure armoring on client computers.

Configuring Armoring for the KDC:

Open Group Policy Management Console and navigate to:

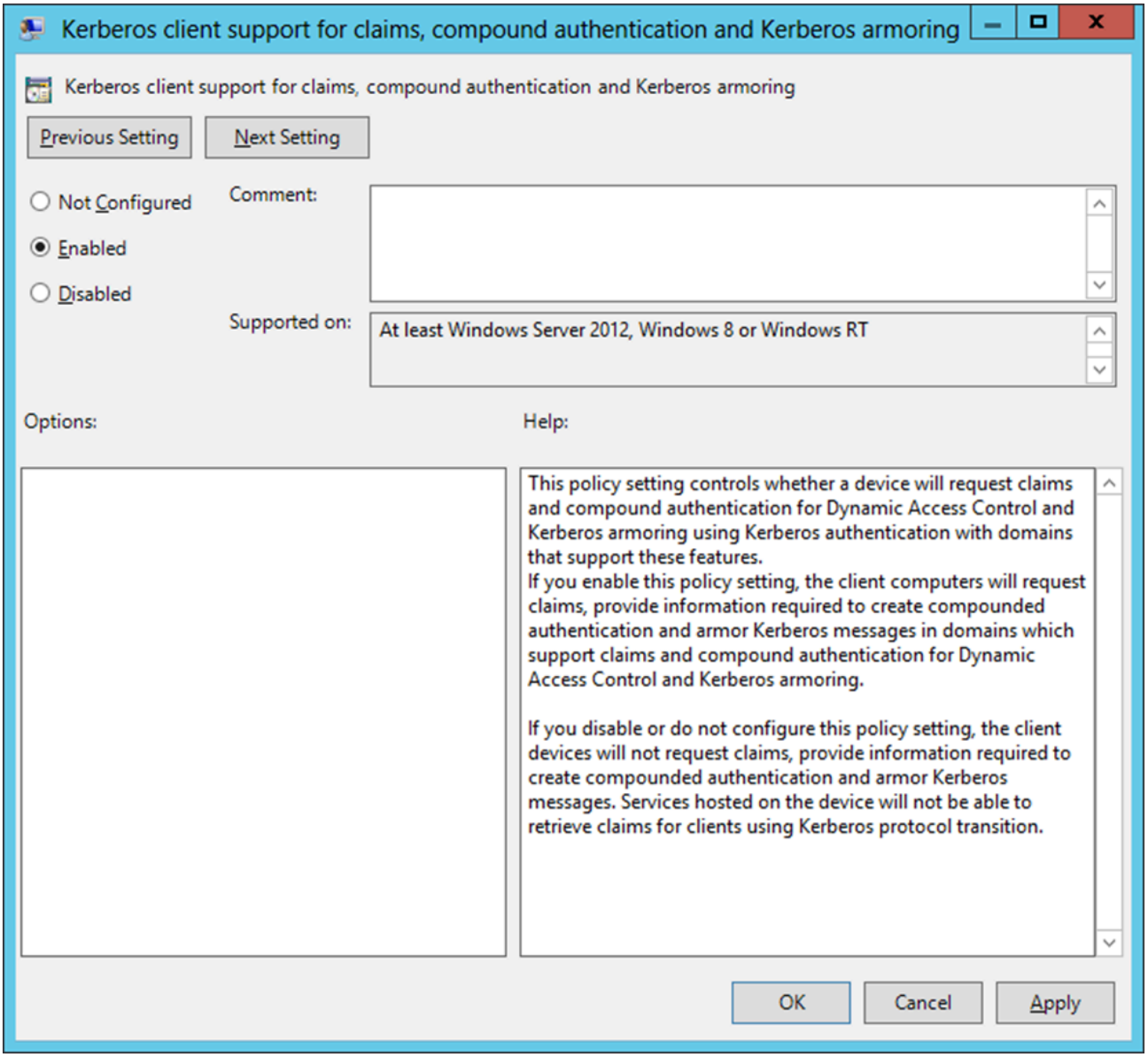
Computer Configuration -> Policies -> Administrative Templates -> System -> KDC



Configuring Armoring for the clients:

Open GPMC and navigate to:

Computer Configuration -> Policies -> Administrative Templates -> System -> Kerberos



The following table shows the various settings available for Kerberos Armoring on the KDC. This should be configured to best suit both operational and security requirements.

Configuration	Results	Domain controller behavior in Windows Server 2012
Not supported (default)	No minimum requirement for domain controllers running Windows Server 2012	Claims not provided Compound authentication not supported Kerberos armoring not supported

Supported	All domain controllers advertise support for claims and compound authentication for Dynamic Access Control and Kerberos armoring Requires sufficient domain controllers running Windows Server 2012 to handle the authentication requests for devices running Windows 8 in the domain	Claims provided on request Compound authentication provided on request when resource supports it Kerberos armoring supported
Always provide claims	All domain controllers advertise support for claims and compound authentication for Dynamic Access Control and Kerberos armoring Requires Windows Server 2012 domain functional level	Claims always provided Compound authentication provided on request when resource supports it Kerberos armoring supported and Flexible Authentication via Secure Tunneling (RFC FAST) behavior supported
Fail unarmored authentication requests	All domain controllers advertise support for claims and compound authentication for Dynamic Access Control and Kerberos armoring Requires Windows Server 2012 domain functional level Requires that all FAST-aware devices request authentication	Claims always provided Compound authentication provided on request when resource supports it Rejects unarmored Kerberos messages and supports the Flexible Authentication via Secure Tunneling (RFC FAST) behavior

Wrap-up

So, can you say you have Kerberos Armoring enabled in your environments? If not, you should and here's why:

1. Kerberos Armoring in Active Directory adds security to some of the more vulnerable parts of Kerberos ticket exchange.

2. Armoring has been available since Windows Server 2012 and is quite simple to implement.
3. Implementing this security control should be well planned and rolled out in groups as armoring can break authentication between systems.
4. It’s 2024, and attacks against Active Directory are becoming more sophisticated by the day. Happy Hunting.